

```

• from keras.layers import Input
• from keras.layers import Dense
• visible = Input(shape=(2,))
• hidden = Dense(2)(visible)
• visible = Input(shape=(2,))
• hidden = Dense(2)(visible)
• model = Model(inputs=visible, outputs=hidden)
• visible = Input(shape=(10,))
• hidden1 = Dense(10, activation='relu')(visible)
• hidden2 = Dense(20, activation='relu')(hidden1)
• hidden3 = Dense(10, activation='relu')(hidden2)
• output = Dense(1, activation='sigmoid')(hidden3)
• model = Model(inputs=visible, outputs=output)
• # summarize layers
• print(model.summary())
• # plot graph
• plot_model(model, to_file='multilayer_perceptron_
graph.png')
• model.compile(Adam(lr=0.05), 'binary_crossentropy',
metrics=['accuracy'])
•
• model_eval = model.evaluate(X_test, y_test)
• plot_model(model, to_file="simple_keras_model.png",
show_layer_names=True, show_shapes=True)
• metadata_1 = y_classifier + np.random.gumbel(scale =
0.6, size = n_row)
• metadata_2 = y_classifier - np.random.laplace(scale =
0.5, size = n_row)
• metadata = np.array([metadata_1, metadata_2]).T
•
• # Create training and test set
• metadata_train = metadata[idx_train,: ]
• metadata_test = metadata[idx_test,: ]
• input_dat = Input(shape=(3,)) # for the three columns of
dat_train
• n_net_layer = Dense(50, activation='relu') # first dense
layer
• x1 = n_net_layer(input_dat)
• x1 = Dropout(0.5)(x1)
•
• input_metadata = Input(shape=(2,))

```